



A paid AI inference oracle for BOT Chain

Whitepaper v0.1 · September 2026 · status: live on BOT Chain Testnet (chainId 968) · [MIT license](#) ·

Contents

- [1. Abstract](#)
- [2. Problem](#)
- [3. Protocol design](#)
- [4. On-chain contracts](#)
- [5. Off-chain components](#)
- [6. Event delivery](#)
- [7. Trust model](#)
- [8. Fees and monetization](#)
- [9. Demand bootstrap: Sentinel](#)
- [10. Limitations and risks](#)
- [11. Roadmap](#)
- [12. Deployments](#)
- [13. Integration](#)

1. Abstract

BOT Oracle is an AI inference oracle deployed on BOT Chain. A consumer contract or wallet submits a paid request naming a model and an input; a staked off-chain operator runs the inference; the result and its output hash are written back on-chain, atomically invoking a consumer callback when one is set. The protocol escrows each fee, pays the operator at fulfillment, and retains a 10% protocol cut. On testnet the system has served hundreds of requests end to end, including a first-party autonomous consumer, Sentinel, that pays for a market-intelligence report every few minutes. The oracle turns "AI on-chain" from a roadmap item into a billable, verifiable service.

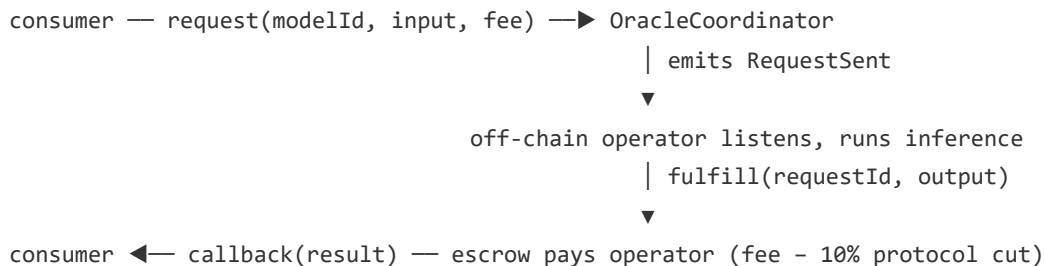
2. Problem

Smart contracts cannot call language models. Any application that wants summarization, classification, or generation must bridge off-chain compute to on-chain state, and today BOT Chain has no oracle infrastructure to do it: no Chainlink, no OAO, no inference router. Every project would otherwise build and operate this layer alone.

The whitepaper for BOT Chain names AI agents and oracle services as core ecosystem components. BOT Oracle delivers that component as running software rather than a plan: deployed contracts, a working operator, a paying consumer, and a public dashboard.

3. Protocol design

Four actors: the consumer contract, the coordinator, the operator, and the model registry.



A request escrows its fee at submission. The operator fulfills within a deadline or the fee refunds via `refundIfTimedOut`. Fulfillment stores the output hash and payload on-chain and calls `callbackContract.onOracleResult(requestId, output)` in the same transaction, so consumers read results atomically.

4. On-chain contracts

Four contracts, all source-verified on the explorer. Consumers integrate one interface (`IOracleConsumer`) and call a single payable method.

```
function request(bytes32 modelId, bytes input,
    address callbackContract, uint64 callbackGasLimit)
    external payable returns (uint256 requestId);

function fulfill(uint256 requestId, bytes output,
    bytes32 inputHash, bytes nodeSig) external onlyOperator;

function refundIfTimedOut(uint256 requestId) external;
function challenge(uint256 requestId) external payable;
```

Contract	Role
OracleCoordinator	Fee escrow, request/fulfill lifecycle, callbacks, disputes
ModelRegistry	modelId → price, backend, container hash, active flag
OperatorRegistry	Operator stake, status, stats; enforces minimum stake
Sentinel	First-party consumer; schedules paid requests, stores reports

5. Off-chain components

oracle-node

Polls for `RequestSent` events, routes inputs to model backends, signs results, and calls `fulfill`. Restart-safe: on boot it rescans from its last processed block, so a crash loses no requests. Backends are pluggable: `echo`: for integration testing, `sentinel`: for the data-bearing report pipeline, and an OpenAI-protocol backend (`LLM_*` env config) currently serving Qwen.

gateway

An HTTP API (`POST /v1/query`) that wraps the on-chain round trip for users who want inference without writing a contract. Metered by API key; every response still anchors on-chain.

@bot-oracle/sdk

A small JavaScript client: `request()`, `awaitResult()`, `getResult()`, `verifyResult()` (recomputes the output hash against chain state). Proven end to end on testnet; installable as a repo tarball today, npm registry publish pending.

dashboard

Server-rendered network view at bot-oracle.druxamb.dev: live request feed, fees accrued, operator stats, and the latest Sentinel report.

6. Event delivery

Probed against live RPCs rather than assumed from docs: no public WebSocket endpoint exists on either network, while `eth_getLogs` works on both despite documentation claiming otherwise. The node therefore uses `getLogs` polling as the primary path with a block-receipt fallback, and the project can run its own node (BOT Chain publishes deploy scripts) if public endpoints degrade.

7. Trust model

v1 is honest about its assumption: a single staked operator, slashable on proven fraud, pinned to a published model hash, with a posted-bond challenge path forcing re-execution. The decentralization ladder is designed so the verification layer can deepen without breaking consumers.

Phase	Model	Guarantee
v1 (now)	Single staked operator + challenge bond	Staked, slashable, replayable
v2	N-of-M operators, threshold agreement	Byzantine tolerance for single faults
v3	TEE attestation / opML fraud proofs	Verifiable inference

8. Fees and monetization

Every request is paid. The coordinator escrows the fee, releases 90% to the operator at fulfillment, and accrues 10% as protocol revenue; the split is verified on-chain, not just documented. Pricing is per-model (testnet: 0.001 BOT per query on the live tier), with USDT-denominated pricing and prepaid balances planned for high-frequency consumers. Off-chain revenue comes through the metered gateway; B2B dedicated operator slots are a later phase.

9. Demand bootstrap: Sentinel

An oracle with no consumers is a demo. Sentinel is the first-party consumer that makes this a business: an autonomous contract that pays for a market-intelligence report on a fixed interval, runs it through the real LLM backend, and stores the result on-chain where the dashboard renders it. Every tick is a request transaction, a fulfill transaction, and a callback, generated by the product itself and auditable on the explorer. 250+ ticks and counting on testnet.

10. Limitations and risks

- Single operator. v1 trusts one staked operator. Challenge bonds and deterministic replay bound the risk; multi-operator is the next phase.
- Operator infrastructure. Node and gateway currently run on a single machine with a self-healing watchdog; durable VPS hosting is planned and gated on ecosystem funding.
- RPC fragility. One public endpoint is a single point of failure; running a dedicated node remains the fallback.

Thin BOT liquidity. On-chain price feeds are unreliable at size (~\$6k real DEX depth when probed), so pricing uses admin-set and USDT-denominated rates rather than spot conversion.

Revenue horizon. On-chain query fees need external consumers; until they exist, Sentinel demonstrates demand and the gateway provides the nearer revenue path.

11. Roadmap

Phase	Deliverable
Done	Testnet protocol, operator, gateway, SDK, Sentinel, live dashboard
Soon	VPS for node + gateway with public endpoint (gated on ecosystem funding, no fixed date); npm registry publish
Next	Second operator on separate infra; challenge and slashing exercised on testnet
Later	Mainnet deployment after audit; TEE/opML verifiable inference; consumer prepaid vaults

12. Deployments

BOT Chain Testnet, chainId 968. All contracts source-verified on the explorer.

Contract	Address
OracleCoordinator	0x4861Ff97A82436d64514C0B119c4796F46a4d8Da
ModelRegistry	0xb208fb3016c14b0946bf3FBbe1Def28d72F63193
OperatorRegistry	0xf22dA276EAA3c4de433115a95111907A6338D3A5
Sentinel (consumer)	0x1ea8e8429Ecae0Dfa8dEbb93983DDe93EA31a014
OracleCoordinator v1 (superseded)	0x7F7e5256cA568B981e1a09642d8F756D9c89F706

13. Integration

Contracts implement `IOracleConsumer` and call `request()` with payment. Scripts use the SDK:

```
$ npm i https://raw.githubusercontent.com/DruxAMB/bot-oracle/main/dist/bot-oracle-sdk-0.1.1.tgz
```

```
import { OracleClient } from "@bot-oracle/sdk";
const o = new OracleClient({ rpcUrl, chainId: 968, coordinator, models, signer });
const value = await o.priceOf(OracleClient.modelId("echo:v1"));
const { requestId } = await o.request({ modelId: OracleClient.modelId("echo:v1"), prompt: "hello", value });
await o.awaitResult(requestId);
const text = await o.getResult(requestId);
```

Self-hosted HTTP gateway: clone the repo, run gateway/, then POST /v1/query with an issued API key. Source, spec, and smoke tests: github.com/DruxAMB/bot-oracle.